B-Tree Indexes Explained

fan-out, range scans, write cost and UUIDv4 keys

Watch the 4-min video

In one line: a B-tree keeps keys sorted in a shallow, balanced tree of pages with huge fan-out, so a lookup in 100 million rows reads 3–4 pages, and linked leaves make range scans cheap; every index makes every write slower.

1 From 100 million rows to four pages

A user opens *My orders*. The orders table has **100 million rows**. Without an index, the database reads all of them to find this user's 20 latest orders. With the right index, it reads about **four pages**. That difference is the B-tree.

Most relational databases (Postgres, MySQL InnoDB, SQL Server, Oracle) index with a **B+ tree**: a balanced tree of fixed-size **pages**, typically **8 KB** (Postgres) or **16 KB** (InnoDB).

- **Internal pages** hold keys and pointers to child pages. They only steer the search.
- **Leaf pages** hold all keys in sorted order, each with a pointer to its row. Every leaf is **linked** to its neighbours.
- The tree is **balanced**: every lookup travels the same distance from root to leaf.

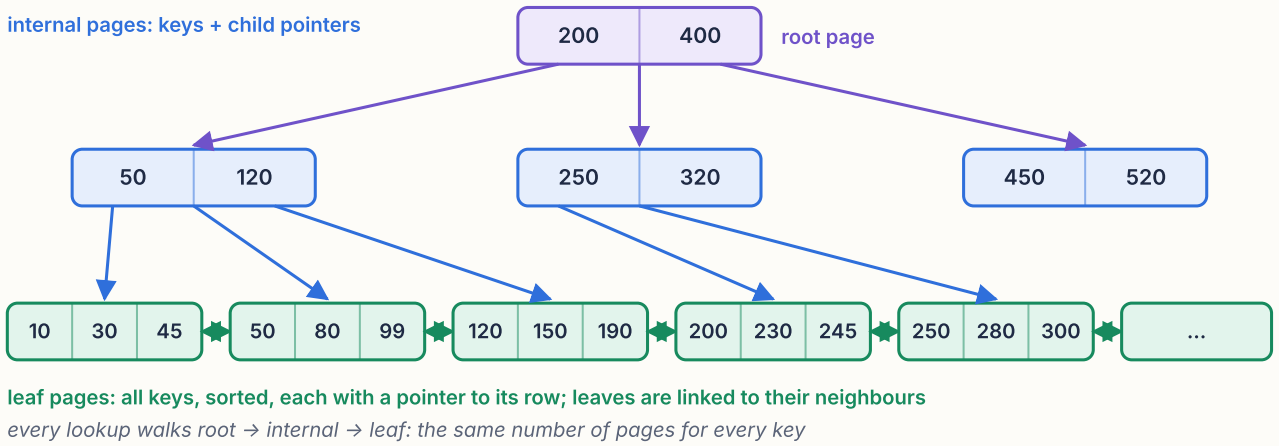


Figure 1. A small B+ tree. To find key 230: root says 200–400 → middle internal page says < 250 → leaf [200, 230, 245]. Three page reads.

2 The magic is fan-out

One page holds hundreds of keys, so each level multiplies the reach by hundreds. That number is the **fan-out**.

$$\text{keys reachable} = \text{fan-out}^{\text{levels}} \quad \cdot \quad 500^3 = 125{,}000{,}000$$

So a lookup among 100 million rows touches only **3 or 4 pages**. The top levels are tiny and used by every query, so they are almost always cached in memory. In practice a lookup costs about one disk read, often zero.



root and level 2 (501 pages, about 4 MB at 8 KB) stay cached, so a lookup costs about one disk read

Figure 2. Fan-out of 500: three levels reach 125 million keys. A fourth level would reach 62.5 billion. Tree depth grows with the logarithm of the row count, so it barely moves as data grows.

3 Range scans and sorted reads

Because leaves are sorted and linked, a **range scan** is cheap: find the first key, then walk along the leaves until the range ends. So a B-tree serves much more than exact lookups:

- **Equality:** `WHERE user_id = 42`
- **Ranges:** `WHERE created_at >= '2026-10-01'`, `BETWEEN`
- **ORDER BY** on the indexed columns, with no sort step, in either direction
- **Prefix matches:** `LIKE 'tap%'` (but not `LIKE '%tap'`)

WORKED EXAMPLE

Index the orders table on **(user_id, created_at)** and run:

```
CREATE INDEX idx_orders_user_time ON orders (user_id, created_at);

SELECT id, total, created_at FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

The database descends to user 42's entries, which are **already sorted by time**, reads 20 of them, and stops: about 3 index pages plus up to 20 row fetches, versus 100 million rows scanned and sorted. If the index also holds every column the query needs (here: also id and total), it becomes an **index-only scan** and the table is not touched at all.

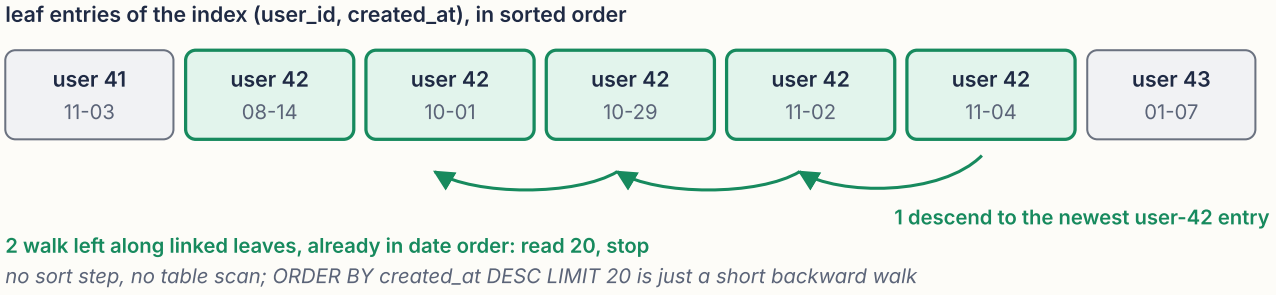


Figure 3. Inside the composite index, all of user 42's orders sit next to each other in date order. The query is a descent plus a short walk.

4 Indexes are not free

- **Every write updates every index.** Each insert, update of an indexed column, or delete must also change each index. A table with **5 indexes** does roughly **6 writes per insert**: one for the row, one per index.

- **Page splits.** When a leaf page fills up, it splits in two. That costs extra I/O and leaves both halves partly empty.
- **Disk and memory.** Indexes take space on disk and compete with table data for the cache.

So index for your **real queries**, and drop indexes nobody uses (in Postgres, `pg_stat_user_indexes.idx_scan = 0` over weeks is a strong hint).

5 Why random UUIDv4 keys hurt

A primary key is itself a B-tree index. With **UUIDv4**, every new key is random, so it lands at a random spot in the tree. Inserts touch random pages all over the index: constant **page splits**, poor **cache hit rates** (the hot set is the whole index), and a **bloated** index with half-empty pages.

Sequential keys, such as a `BIGINT` identity, or **time-ordered IDs** like **UUIDv7** (a 48-bit millisecond timestamp, then random bits), always append near the **right edge** of the tree. Writes hit the same few cached pages, and pages stay full.

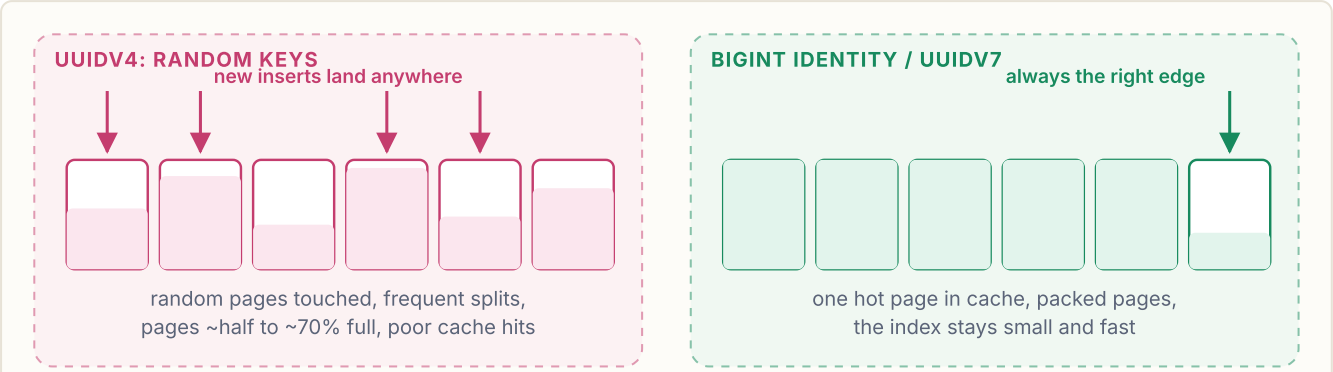


Figure 4. Left: random keys spread writes over every leaf. Right: increasing keys fill one page at a time at the right edge. UUIDv7 keeps global uniqueness without the random-insert penalty.

COMMON MISTAKES

- Indexing every column “just in case”: each one taxes every write.
- Using random UUIDv4 as the clustered primary key on a high-insert table.
- Expecting an index to help `LIKE '%term'` or a function on the column, such as `WHERE lower(email) = …` on a plain email index.
- Indexing a column with low selectivity (a boolean that is true for 90% of rows) and expecting the planner to use it.

6 Key terms

B+ tree: balanced tree of pages; all keys in linked leaves

Fan-out: child pointers per page, often hundreds

Index-only scan: answered from the index, table untouched

Write amplification: one row write becomes many page writes

Page: fixed-size block, 8 KB or 16 KB

Range scan: find first key, walk the leaves

Page split: full page divided in two on insert

UUIDv7: time-ordered UUID; appends at the right edge

7 Interview questions

Q1. How does a B-tree index speed up queries? BASIC

It keeps keys sorted in a shallow, balanced tree with high fan-out. A lookup in 100 million rows touches only 3–4 pages instead of scanning the table, and range scans walk the linked leaves in order.

Q2. What is the cost of too many indexes? BASIC

Every write must update each index (5 indexes ≈ 6 writes per insert), page splits add I/O, and indexes use disk and cache memory. Writes slow down and storage grows.

Q3. Why is a B-tree only 3–4 levels deep even for huge tables? BASIC

Fan-out. With about 500 keys per page, 3 levels reach 500^3 = 125 million keys and 4 levels reach 62.5 billion. The top two levels are small enough to stay in memory.

Q4. "My orders" is slow on a 100M-row table: WHERE user_id = ? ORDER BY created_at DESC LIMIT 20. What index? SCENARIO

A composite index on (user_id, created_at). The database jumps to that user's entries, already in time order, reads 20 and stops: no sort, no scan. Add the selected columns (or INCLUDE them) for an index-only scan.

Q5. Insert throughput on an events table dropped as it grew. The PK is UUIDv4. Why, and what do you change? SCENARIO

Random keys insert into random leaf pages, so once the index outgrows memory every insert reads a cold page, splits are frequent and pages run half empty. Switch to a BIGINT identity or UUIDv7 so inserts append at the right edge.

Q6. Will an index on created_at help WHERE created_at > now() - interval '5 years'? TRICKY

Probably not, if that matches most of the table. For low-selectivity filters, a sequential scan is cheaper than millions of random index-to-row hops, and the planner will choose it.

Q7. Can a B-tree index on (user_id, created_at) serve ORDER BY created_at DESC for one user? TRICKY

Yes. Leaves are linked both ways, so the database can walk backward from the newest entry. Within one user_id, entries are already in created_at order, so no sort is needed.

Next: SD-E-005 · Composite, Covering & Partial Indexes backendroadmap.com